

{Insert Title Here}: Fine-tuning a language model to predict titles

1st Louis Stanko

Ludwig-Maximilians-Universitaet

Munich, Germany

l.stanko@campus.lmu.de

Abstract—This paper presents a partial replication study of "Can Pre-Trained Language Models Generate Titles for Research Papers?" Instead of fine-tuning large or medium-scale transformer models, I investigate whether a 135-million parameter model, called SmolLM-135M, can achieve competitive title generation performance. I experiment with both full fine-tuning (FFT) and Low-Rank Adaptation (LoRA) across two training regimes: a small domain-specific dataset (CSPubSum) and a larger, more diverse dataset comprising over 2.4 million samples drawn from arXiv, Wikipedia, and Medium blog articles. Evaluation is performed on the LREC-COLING 2024 benchmark using ROUGE-1, ROUGE-2, and ROUGE-L scores. My best configuration, full fine-tuning on the large dataset, achieves ROUGE-1/2/L scores of 41.32/22.51/35.21, approaching the performance of fine-tuned large language models at a fraction of the parameter count. In the following, I discuss training dynamics, the effect of dataset scale, and the impact of generation length on evaluation scores. Last but not least, the final model is converted to ONNX format and deployed as a browser-based application using Transformers.js.

Index Terms—SmolLM-135M, fine-tuning, title generation

I. INTRODUCTION

Automatic title generation for scientific papers is a fairly well-studied sequence-to-sequence task with broad practical applications like academic search, recommendation systems, or mobile apps like calendars or To-do apps. A well-formed title should be concise, informative, and factually grounded in the source. At the same time, it is constrained to a relatively short output, typically under 20 tokens, which raises an interesting question: does effective title generation actually require a large language model? The reference paper by Rehman et al. [2], "Can Pre-Trained Language Models Generate Titles for

Research Papers?", explores this using the models: T5-base, BART-base, PEGASUS-large, LLaMA-3-8B, and GPT-3.5-turbo on the LREC-COLING 2024 benchmarks. However, the study does not examine the low end of the model size spectrum.

Motivated by the hypothesis that title generation is a relatively constrained task that may not require large-scale parametric capacity, this work investigates SmolLM-135M, a decoder-only causal language model with only 135 million parameters. I conduct experiments across two fine-tuning strategies (full fine-tuning and LoRA) and two dataset regimes (domain-specific and large-scale diverse), and report results on the LREC-COLING 2024 test set. The LREC-COLING 2024 dataset was introduced in the reference paper by Rehman et al. [2]. The goal is not to achieve state-of-the-art performance, but to understand how far a small, efficiently deployable model can go, and to identify the conditions under which it succeeds or fails.

As a secondary contribution, I demonstrate practical deployment: the best model is exported to ONNX format and served entirely in the browser via Transformers.js, illustrating that research-quality title generation can run locally without any server infrastructure.

II. RELATED WORK

Title generation has historically been treated as a summarization problem, where the abstract plays the role of the source document and the title is the extreme summary. Early work used extractive approaches and template-based methods. With the rise of neural sequence-to-sequence models, abstractive generation became dominant, with LSTM-

based encoder-decoders eventually giving way to transformer architectures [1].

The reference paper by Rehman et al. [2] is the most directly relevant work. They benchmark both encoder-decoder models and decoder-only LLMs on CSPubSum and LREC-COLING 2024, finding that fine-tuned T5-base and BART-base yield ROUGE-1 scores around 46–47 percent, while PEGASUS-large reaches nearly 50%. Notably, the non-fine-tuned LLaMA-3-8B model in a zero-shot setting scores only 32.92 on ROUGE-1, highlighting the importance of task-specific fine-tuning even for large models.

LoRA [3] has emerged as an efficient alternative to full fine-tuning by injecting trainable low-rank matrices into frozen transformer layers. Rather than updating all model weights, LoRA introduces a small number of additional parameters (controlled by rank r) that are sufficient to adapt the model to new tasks. This significantly reduces memory and compute requirements, making it well-suited for consumer hardware.

SmolLM [4] is a family of compact causal language models trained on the SmolLM-Corpus, which includes curated web text and code. The 135M variant, used in this work, uses the MobileLLM [5] architecture and cosmo2-tokenizer [6], making it compatible with standard HuggingFace tooling. To my knowledge, no prior work has evaluated SmolLM-135M on title generation.

III. DATA

A. Training Datasets

I used two distinct training corpora representing different scales and domains.

CSPubSum: The CSPubSum training dataset contains 8,120 abstract-title pairs from computer science publications. This dataset was used in the original reference paper and represents the primary in-domain training resource for scientific title generation. Cleaning follows the same protocol as the reference paper: title length is constrained to between 3 and 20 tokens, and abstracts must be at least 20 tokens long. This requirement is necessary, in order to treat the title generation as an extreme form of text summarization.

Large (diverse) Dataset: To test whether broader training signal can compensate for domain mis-

match, I construct a second training corpus by combining three sources: (1) a Kaggle dataset merging arXiv abstracts with Wikipedia articles, totaling approximately 2.65 million entries; (2) the Medium Articles dataset from Kaggle, containing approximately 192,000 blog posts with titles. The two sources are concatenated and subjected to a shared cleaning pipeline. **Note:** In the following this dataset is referred to as *Large Dataset* or *Large Diverse Dataset*. In the python code, this is usually referred to as *own* or *train_data.csv*.

TABLE I
DATASETS USED

Name	Samples
Wikipedia	900,000
Medium.com	190,000
Arxiv	1,750,000

B. Data Cleaning

Both datasets pass through the following cleaning steps. First, duplicates are removed. Second, samples with titles containing LaTeX mathematical expressions (e.g., dollar-sign delimited formulas), numbers greater than 99, or parenthetical expressions are filtered out, as these tend to produce noisy or unpredictable targets. Third, all samples are tokenized using the cosmo2 tokenizer, and only those where the concatenated text (abstract + title prompt) fits within 512 tokens are retained. Finally, any samples whose titles appear in the LREC-COLING 2024 **test set** are removed to prevent data leakage.

After cleaning, the large dataset contains approximately 2.41 million samples. A notable consequence of the LaTeX filtering is the removal of a substantial fraction of arXiv papers, many of which contain mathematical notation in their titles. This may shift the distribution of the large dataset toward more general-domain content, which may explain some of the generalisation patterns observed in the results. See Figure 1 for a detailed breakdown of the cleaning pipeline.

C. Evaluation Dataset

All models are evaluated on the LREC-COLING 2024 test set, a benchmark of scientific paper title-abstract pairs released alongside the reference paper.



Fig. 1. Visualization of data cleaning pipeline for the large diverse dataset.

As an effort to make my results easier to reproduce, I published the test set on Kaggle [7]. This set is held out from all training data, as described above.

IV. METHODOLOGY

A. Base Model

All experiments use SmolLM-135M v1 (HuggingFaceTB/SmolLM-135M) as the base model. This is a decoder-only causal language model with 135 million parameters, trained on a mixture of web text and code. It uses the cosmos2-tokenizer in conjunction with a vocabulary of size 49,152. The model is publicly available on HuggingFace [8]. For more information about the architecture and hyper-parameters used, visit <https://huggingface.co/blog/smollm>.

I decided to use the first version of the SmolLM-135M model, as the successor only yielded small improvements over the first version. Additionally, the first version was released in February 2024, which is close to the release date of the reference paper (September 2024), allowing for a fairer comparison.

B. Prompt Format

I frame title generation as a conditional language modeling task. Each training example is formatted as:

```
<abstract>\n\nTitle: <title><EOS>
```

During inference, only the abstract and the "Title:" prompt are provided, and the model generates the title autoregressively. Loss is computed only on the title tokens (the abstract prefix is masked with -100 labels in the full fine-tuning setup). This causal language modeling approach is standard for decoder-only models adapted to generation tasks. When fine-tuning LoRA, no masking was done and the loss was computed on the entire input sequence. This was due to a lack of understanding. Unfortunately, this could have had a negative impact on fine-tuning, as the model gets rewarded to memorize the entire input sequence rather than just focusing on the title.

C. LoRA Fine-Tuning

The first experimental setup uses Low-Rank Adaptation (LoRA). LoRA is applied with rank (r) = 16, alpha (α) = 16, and dropout of 0.05, targeting the default query and value projection layers. Since $r == \alpha$, the **effective adaptation strength** ($\frac{r}{\alpha}$) = 1, implying a stable adjustment of weight changes. As α denotes the "impact" of the update which is applied to the weights and r The model is trained on the large diverse dataset for 5 epochs using the SFTTrainer. Batch size is 8 with 4 gradient accumulation steps (effective batch size 32), and learning rate is 4e-5 (0.00004). These metrics were taken from the reference paper, but are also considered "sane defaults" in the machine learning community. The SFTTrainer (supervised fine-tuning) operates on raw formatted text strings rather than pre-tokenized sequences, using the text field from the formatted examples, however, during cleaning the actual input size was already measured in tokens.

Also relevant for the LoRA configuration, is that the base model contains eight distinct linear layer types: `q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`, and `lm_head`. By default, the python library used for this experiment, huggingface

PEFT [9], adjusts only `q_proj` and `v_proj`. I adopt this default in the experiments due to computation and time constraints. Nevertheless, I would encourage further experiments which adapt more linear layers as these would likely boost accuracy.

Another observation which is worth mentioning that these settings reduce the size from 135 million (135,436,608 to be exact) parameters to 921,600. That is 0.68% of all tunable parameters and might help explain some of the results in the later sections.

The fine-tuning happened on an NVIDIA V100 16GB VRAM GPU node hosted in the LRZ cloud. Training on the large dataset took 100 hours.

After training, the LoRA adapter is merged into the base model weights, which produces a standard HuggingFace model that can be distributed independently.

D. Full Fine Tuning (FFT)

In the second experimental setup, all model weights are updated during training. I conduct two FFT runs: one on the small CSpUBSum dataset and one on the large diverse dataset.

For the CSpUBSum run, training proceeds for 1 epoch using a batch size of 16 with 2 gradient accumulation steps (effective batch size 32), an unchanged learning rate of $4e-5$ with a cosine scheduler and 3% warmup ratio. For the large dataset run, the same hyperparameters are used but training is capped at 1 epoch due to the dataset size and insights from the previous fine tuning run, where after the first epoch a validation plateau was reached.

The fine-tuning happened on an NVIDIA V100 16GB VRAM GPU node hosted in the LRZ cloud. Training took 10 hours.

E. Evaluation

Title generation is performed with greedy decoding (`do_sample=False`, for repeatability) and a maximum of 30 new tokens. The 30-token limit was selected based on empirical observation: using 20 tokens produced significantly worse scores (approximately 20% ROUGE-1 degradation). An investigation (Figure 2) revealed that, although the distribution of title token lengths has a mean of

15.27 tokens, it exhibits a right tail, with a non-trivial portion of titles exceeding 20 tokens. When generation is capped at 20 tokens, these longer titles are forcibly truncated before completion. This causes the model to output partial titles that fail to match their full references during ROUGE evaluation. A truncated output structurally cannot recover the tokens it never produced, negatively impacting recall and the F1 score. Extending the limit to 30 tokens covers the overwhelming majority of the distribution, allowing nearly all titles to be generated in full.

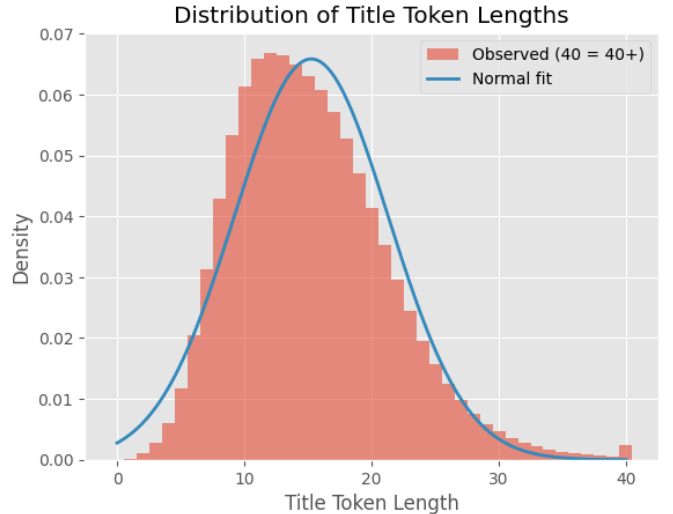


Fig. 2. Average Title Token Distribution.

F. Evaluation Metrics

To make my results comparable a widely accepted baseline called ROUGE is used. ROUGE stands for Recall-Oriented Understudy for Gisting Evaluation, and can be measured in different ways. I report F1 scores for ROUGE-1, ROUGE-2, and ROUGE-L using the `rouge-score` python library with stemming enabled [11]. ROUGE-1/2 are part of the ROUGE-N family which measures the n -gram between a *hypothesis* (the generated title) and the *ground-truth* (the actual title). An n -gram is a sequence of n adjacent symbols in a particular order. For example, Table II illustrates how a sentence is decomposed into 1-grams and 2-grams.

ROUGE-L, simply measures the longest common subsequence (LCS) in both, the hypothesis and the ground-truth. A subsequence is a string generated

TABLE II
EXAMPLE OF n -GRAM EXTRACTION FROM A SAMPLE SEQUENCE

Example	1-gram sequence	2-gram sequence
... to be or not ...	... to, be, or, not, ...	... to be, be or, or not, ...

from the ground-truth by deleting 0 or more characters, without changing the relative order of the remaining characters.

Last but not least, stemming is a preprocessing technique that reduces words to their word stems by removing suffixes and prefixes (e.g. runs $\rightarrow$ run & running $\rightarrow$ run). This might be a deviation from the reference paper, unfortunately, I was unable to find any artifacts of the code used for evaluating Rehman et al. [2] fine-tuned models.

V. RESULTS

ROUGE scores are computed against lowercased and stemmed ground-truth titles using F1 measures for ROUGE-1, ROUGE-2, and ROUGE-L.

Additionally, entity-level factual consistency metrics (precision, recall, F1 over named entities extracted using SciSpacy) are also computed in the evaluation script, following the methodology of the reference paper, though, in the following, I focus on ROUGE as the primary comparison metric due to required brevity.

The results of the factual consistency score can be found in the appendix 8.

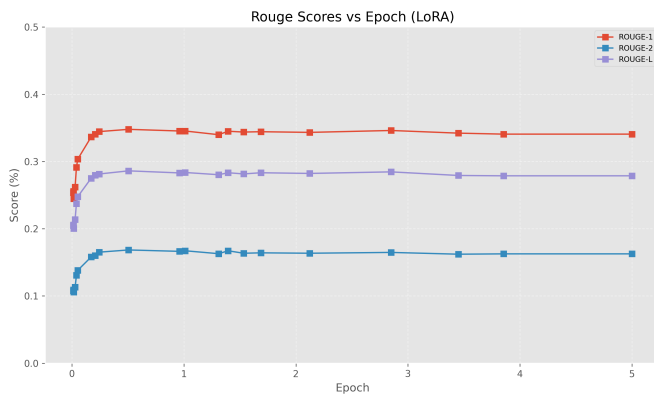


Fig. 3. ROUGE scores of **Large Dataset** when fine-tuning using LoRA for 5 epochs. The validation set was LREC-COLING 2024.

During the first half of the first epoch, rouge scores increase dramatically. From $\approx 25\%$ to $\approx 34\%$ for ROUGE-1, which is a 40% increase! For the rest

of the fine-tuning, performance hits a plateau. For ROUGE-1 that is at around 33%. ROUGE-2 and ROUGE-L hover around 16% and 28% respectively. The graph is capped at 50%, to show the ROUGE-1 score of the best performing model from the reference paper, PEGASUS-large.

The best recorded checkpoint was after 42200 steps. 5 epochs were 417345 steps and took 100 hours walltime. In hind-sight, fine-tuning could have been stopped after ≈ 10 hours. However, validation was done after the fine-tuning finished, revealing this pattern too late.

Table III presents the measurements from the best checkpoint across all experimental configurations alongside the measurements from the reference paper. Our own results are highlighted in bold.

TABLE III
ROUGE-1/2/L F1 SCORES ON THE LREC-COLING 2024 TEST SET. REFERENCE RESULTS FROM REHMAN ET AL. [2]. RESULTS IN BOLD.

Model	ROUGE Metrics		
	<i>1</i>	<i>2</i>	<i>L</i>
T5-base	46.84	28.70	41.69
BART-base	46.87	27.66	41.89
PEGASUS-large	49.85	30.51	43.93
LLaMA-3-8B zero-shot	32.92	16.66	27.66
LLaMA-3-8B fine-tuned	45.30	26.53	40.51
GPT-3.5-turbo	45.16	23.97	38.88
SmolLM LoRA CSPubSum	25.52	10.90	20.54
SmolLM FFT CSPubSum	35.36	17.54	30.12
SmolLM LoRA Large Dataset	34.79	16.84	28.61
SmolLM FFT Large Dataset	41.32	22.51	35.21

^a ROUGE scores reported as percentages.

One can see, the FFT model trained exclusively on CSPubSum (8,120 samples) achieves ROUGE-1/2/L of 35.36/17.54/30.12. While this is below almost all reference models, it is notable that a 135M parameter model fine-tuned on fewer than 10,000 samples produces non-trivial scores at all, suggesting the task is learnable even with minimal data.

The LoRA model trained on the same CSPubSum dataset achieves lower scores of 25.52/10.90/20.54. This is likely due to the fact that the base model lacked exposure to scientific corpora during pre-training. Therefore, the restricted parameter space of the adapters cannot effectively synthesize the necessary domain-specific knowledge.

In contrast, the LoRA model trained on the large diverse dataset for 5 epochs achieves 34.79 / 16.84 / 28.61, a slight regression from the CspNetSum FFT baseline. This is likely due to the limited representational flexibility of low rank adaption. Here, scientific summarization could simply be too complex and benefit from deeper model adaption.

The strongest result is achieved by the FFT model trained on the large diverse dataset: 41.32/22.51/35.21. This is competitive with the zero-shot LLaMA-3-8B baseline (32.92/16.66/27.66) and comes close to models that are 60 times larger. The gap to the best reference model (PEGASUS-large: 49.85/30.51/43.93) remains substantial, but the result illustrates that a 135M model with appropriate training data can perform surprisingly well.

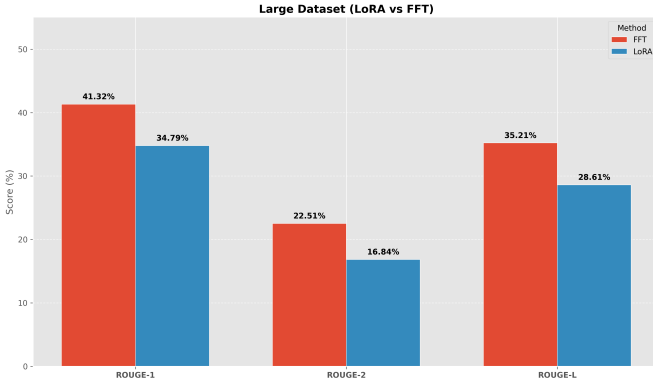


Fig. 4. Best ROUGE scores of **Large Dataset** between LoRA (5 epochs) and FFT (1 epoch). The validation set was LREC-COLING 2024.

In Figure 4 & 5, one can see that full fine-tuning always outperforms LoRA by a notable margin. Why this could be is discussed in the following chapter.

VI. DISCUSSION

A. Dataset Scale vs. Domain Specificity

My biggest finding is that the large diverse dataset does not always outperform the in-domain CspNetSum dataset, despite the former still containing substantially more scientific content. This suggests that, for a model as small as SmoLLM-135M, learning the general task of summarizing text into a short, descriptive headline from a large and varied corpus may be too much and more confusing than learning

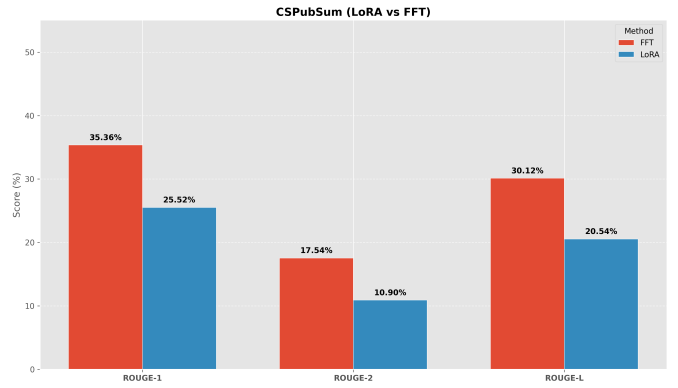


Fig. 5. Best ROUGE scores of **CspNetSum** between LoRA (5 epochs) and FFT (1 epoch). The validation set was LREC-COLING 2024.

the narrow stylistic conventions of scientific titles (underfitting). Larger models, by contrast, may be better positioned to exploit in-domain data efficiently.

B. Overfitting

A potential weakness of the large dataset experiment is the risk of overtraining. Although the diverse corpus contains over 2.4 million samples after cleaning, the title length distribution differs substantially from the LREC-COLING evaluation set, which focuses on scientific papers. During the LoRA training (5 epochs), performance was observed to plateau after the first epoch on held-out validation loss. This is consistent with a form of distribution shift: the model adapts increasingly to the style of Medium blog titles and general web content, drifting away from the scientific register expected by the benchmark.

The large-scale FFT experiment was restricted to a single epoch to avoid this degradation. Even so, it is possible that the model has not fully converged, and that a more carefully tuned learning rate schedule or earlier stopping criterion could yield further improvements.

C. Impact of Generation Length

The choice of maximum generation length (`max_new_tokens`) has a disproportionate effect on ROUGE scores in this setting. Restricting generation to 20 tokens approximately halved ROUGE-1 performance compared to 30 tokens. This is an artifact of the training data distribution: the large

dataset contains many titles substantially longer than those in CSPubSum, so the model learns to generate longer outputs. When the generation is cut short, partially generated titles receive poor ROUGE scores.

This is a methodological weakness relative to the reference paper, which evaluates models trained on domain-matched data with more consistent title lengths. If in the future further research is done, it work should consider normalizing title length either during training by filtering for length consistency or during evaluation, using length-normalized ROUGE variants.

D. LoRA vs. Full Fine-Tuning

At this model scale, full fine-tuning outperforms LoRA by a notable margin (41.32 vs. 35.36 ROUGE-1). This is somewhat expected: with only 135M parameters, the model is small enough that full fine-tuning remains computationally feasible, and the flexibility of updating all weights allows the model to better adapt to the target task. The advantage of LoRA lies primarily in memory efficiency and faster training iteration, which becomes more relevant for larger models. At 135M parameters, the memory savings are less critical.

E. Limitations

Several limitations should be acknowledged. First, the evaluation is restricted to ROUGE scores. I could not go into detail to report the full entity-level factual consistency metrics from the reference paper, since this would extend the paper by a notable margin. However, these can be found in the appendix.

Second, the LREC-COLING 2024 test set evaluates performance on computer science papers, but the large training dataset is heavily weighted toward general web content after the LaTeX filtering step, creating a domain mismatch that may suppress performance.

Third, no hyperparameter search was conducted; the learning rates, batch sizes, and LoRA rank were chosen based on reasonable defaults rather than systematic optimization.

Fourth, and most fundamentally, SmoILM-135M is a decoder-only causal model, whereas the best-performing reference models (T5, BART, PEGA-

SUS) are encoder-decoder architectures specifically designed for conditional generation. This architectural difference may inherently disadvantage our approach for short, precise generation tasks. This can also be seen when comparing the ROUGE scores visually (Figure 6). All decoder & encoder models dominate the benchmarks, even compared to much bigger decoder only models like the 8 billion Llama 3 model.

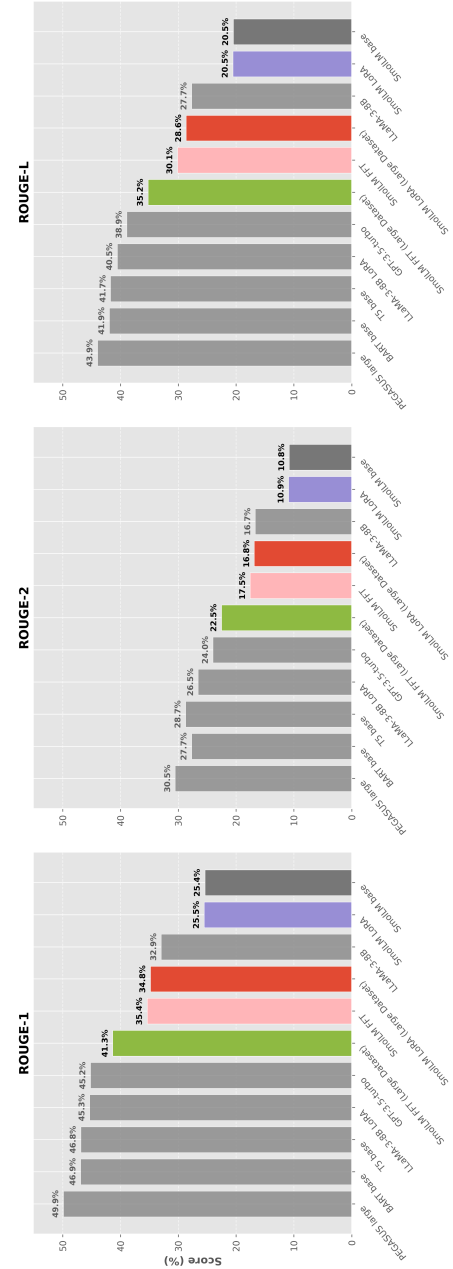


Fig. 6. Best ROUGE Scores of all models, compared visually. All SmoILM models are color coded.

VII. PRACTICAL DEPLOYMENT: TITLE GENERATION IN THE BROWSER

As a practical contribution beyond the numerical results, the LoRA adapters were merged back into the base model was converted to ONNX format using the Optimum library and subsequently deployed as a browser-based application using Transformers.js. ONNX runtime allows model inference to run entirely within the browser via WebAssembly, without requiring any API calls (apart from downloading the model into local storage once).

The pipeline for deployment is as follows: the LoRA adapter checkpoint is first merged into the base model weights, producing a standard HuggingFace model. This model is then exported to ONNX. The resulting model file, along with the tokenizer configuration, is saved and hosted on HuggingFace Hub under the identifier `throvn/abstract-title-generator`. The live demo is accessible at:

<https://throvn.github.io/abstract-title-generator-frontend/>

Users can paste any scientific abstract (or other text) into the interface and receive a generated title directly in the browser. The model runs locally on the user's machine after the initial download.

VIII. CONCLUSION

This work presented a partial replication and extension of the paper "Can Pre-Trained Language Models Generate Titles for Research Papers?" using SmolLM-135M, a 135-million parameter language model. I conducted experiments across two fine-tuning strategies and two dataset regimes, finding that the combination of full fine-tuning and a large diverse training corpus yields ROUGE-1/2/L scores of 41.32/22.51/35.21 on the LREC-COLING 2024 benchmark. This result is competitive with zero-shot LLaMA-3-8B and within reasonable range of fine-tuned models an order of magnitude larger (like ChatGPT-3.5-turbo).

My findings support the hypothesis that title generation, being a relatively constrained and short-form generation task, does not necessarily require large model capacity. The primary bottleneck at the 135M scale appears to be training data quality rather than scale or model size.

TITLE GENERATION · RESEARCH · LMU

Abstract *Title* Generator

Powered by `Throvn/abstract-title-generator` - a fine-tuned SmolLM-135M.
Paste your abstract and the model will infer an appropriate title.

● Done

ABSTRACT 1687 CHARS

Studies (KISS) workshop on PE 3.0, covering: (1) PE 3.0 systems engineering including science definition, architecture, design methods, and verification & validation; (2) software-defined space system technologies including reconfigurable hardware, multi-functionality, and modularity; (3) onboard intelligence including autonomous science, navigation, controls, and embodied AI; and (4) three PE 3.0 mission concepts: a Neptune/Triton smart flyby, an ocean world explorer, and an Oort cloud reconnaissance mission.

Transformer attention Climate ML Medical imaging

MAX NEW TOKENS 30 TEMPERATURE 0.70
TOP-P (NUCLEUS) 0.90 REPETITION PENALTY 1.20

GENERATE TITLE CLEAR

GENERATED TITLE

Why not just explore? An approach towards planetary exploration without stepping into unknown territory ?

COPY TITLE

Model: `Throvn/abstract-title-generator` · In-browser inference via `Transformers.js`
· Made with passion by `Throvn`

Fig. 7. Online hosted abstract title generator

IX. DATA & CODE

Since the resources are too big to submit to moolle, I host them on github, kaggle, and huggingface. The resources are available here:

- Source Code
- LREC-COLING 2024 Dataset
- Finetuned LoRA Model (transformers.js compatible)

X. APPENDIX

TABLE IV

FACTUAL CONSISTENCY SCORES FOR NON UNIQUE ENTITIES

Model	NU Target Metrics		
	Precision	Recall	F1
T5-base	63.06	59.24	57.17
BART-base	64.30	59.21	57.78
PEGASUS-large	66.32	64.64	61.47
LLaMA-3-8B*	45.29	48.16	43.48
LLaMA-3-8B	62.18	55.99	55.19
GPT-3.5-turbo	60.87	57.66	55.56
SmolLM-135M FFT Large Dataset	51.13	38.90	41.68
SmolLM-135M FFT CspNetSum	41.53	35.11	35.75
SmolLM-135M LoRA Large Dataset	36.89	53.20	41.37
SmolLM-135M LoRA CspNetSum	24.61	36.15	27.93

* Zero-shot evaluation. Metrics reported as percentages.

TABLE V

FACTUAL CONSISTENCY SCORES FOR UNIQUE ENTITIES

Model	U Target Metrics		
	Precision	Recall	F1
T5-base	62.80	58.88	56.87
BART-base	64.11	58.92	57.56
PEGASUS-large	66.15	64.30	61.28
LLaMA-3-8B*	45.02	47.64	43.13
LLaMA-3-8B	62.13	55.95	55.16
GPT-3.5-turbo	60.84	57.69	55.56
SmolLM-135M FFT Large Dataset	51.02	38.70	41.51
SmolLM-135M FFT CspNetSum	41.41	34.68	35.51
SmolLM-135M LoRA Large Dataset	35.95	46.88	38.79
SmolLM-135M LoRA CspNetSum	24.18	34.13	26.98

* Zero-shot evaluation. Metrics reported as percentages.

REFERENCES

- [1] Sutskever, Ilya, Oriol Vinyals, and Quoc V. Le. "Sequence to sequence learning with neural networks." *Advances in neural information processing systems* 27 (2014).
- [2] Rehman, Tohida, Debarshi Kumar Sanyal, and Samiran Chattopadhyay. "Can pre-trained language models generate titles for research papers?." *International Conference on Asian Digital Libraries*. Singapore: Springer Nature Singapore, 2024.

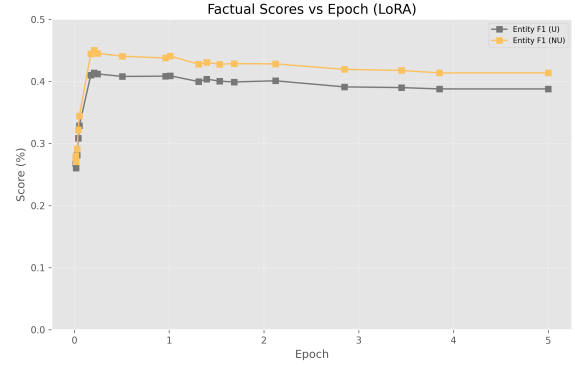


Fig. 8. Online hosted abstract title generator

- [3] Hu, Edward J., et al. "Lora: Low-rank adaptation of large language models." *Iclr* 1.2 (2022): 3.
- [4] HuggingFace, "SmolLM - blazingly fast and remarkably powerful" 2024. [Online]. Available: <https://huggingface.co/blog/smollm>
- [5] Z. Liu, C. Zhao, F. Iandola, C. Lai, Y. Tian, I. Fedorov, Y. Xiong, E. Chang, Y. Shi, R. Krishnamoorthi, L. Lai, and V. Chandra, "MobileLLM: Optimizing Sub-billion Parameter Language Models for On-Device Use Cases," *arXiv preprint arXiv:2402.14905*, 2024. [Online]. Available: <https://arxiv.org/abs/2402.14905>
- [6] Hugging Face, "cosmo2-tokenizer: Tokenizer for the training of cosmo2," 2024. [Online]. Available: <https://huggingface.co/HuggingFaceTB/cosmo2-tokenizer>
- [7] Thrown, "LREC-COLING 2024: Collection of datasets from the Joint International Conference on Computational Linguistics, Language Resources and Evaluation," 2024. [Online]. Available: <https://www.kaggle.com/datasets/thrown/lrec-coling-2024>
- [8] Hugging Face, "SmolLM-135M: A 135M-parameter small language model from the SmolLM family," 2024. [Online]. Available: <https://huggingface.co/HuggingFaceTB/SmolLM-135M>
- [9] Hugging Face, "PEFT (Parameter-Efficient Fine-Tuning): Documentation for efficient adaptation of large pretrained models," 2024. [Online]. Available: <https://huggingface.co/docs/peft/index>
- [10] Lin, Chin-Yew. "Rouge: A package for automatic evaluation of summaries." *Text summarization branches out*. 2004.
- [11] Google LLC, "rouge-score: Pure Python implementation of ROUGE-1.5.5," 2019. [Online]. Available: <https://pypi.org/project/rouge-score/>